Design of a Traffic Light Controller Using Behavioral Verilog HDL

Bryce Kelly & Dylan Thornburg

Abstract
This paper discusses a behavioral Verilog HDL implementation of a traffic light controller for a four-way intersection. The design supports three operating modes: normal traffic, increased traffic with configurable duty-cycle ratios, and an emergency mode. Each mode is implemented as an independent finite state-machine (FSM) described at the RTL level using synthesizable behavioral constructs. A top-level wrapper module integrates all three modes with a mode-select input that allows for switching between them. The designs are synthesized using Synopsys Design Compiler targeting the lsi_10k.db dw_foundation.sldb cell library. This work improves upon the gate-level combinational approach presented by Zhilevski et al. by replacing it with explicit state machines and a unified top-level controller. Simulation results from Synopsys VCS confirm correct operation across all modes, and synthesis reports are used to compare area, power, and timing characteristics.

Introduction
Traffic signal control is a well-studied and commonly taught problem in digital design, and is frequently used as an example for finite state machines. A real intersection controller must handle multiple operating conditions such as standard timed cycling, adaptive responses to congestion, and a fallback mode when the system is disabled or malfunctioning. We can implement these behaviors quite easily in a hardware description language such as Verilog which can be used on real hardware such as FPGAs.

Zhilevski et al. [1] presented a Verilog implementation of a traffic light system for a two-road intersection with eight lights (four vehicle, four pedestrian) and three operating modes. Their design used gate-level primitives to construct the control logic combinationally, with separate Verilog source files for each mode and no unified top-level controller. This approach has several limitations. The absence of explicit state encoding makes the design more difficult to understand and verify, the combinational structure risks latch inference if not carefully managed, and the lack of a top-level wrapper means mode switching must be handled externally.

This project addresses those limitations by reimplementing all three modes as behavioral RTL state machines with explicit state encoding, and clean separation between sequential and combinational logic. A top-level module is introduced that multiplexes between the three mode FSMs based on a two-bit select input, enabling runtime mode transitions. The designs are synthesized with no inferred latches.

Background
The intersection at hand consists of two perpendicular roads, each with two lanes. Eight traffic lights are placed at the intersection: four for vehicles (T11, T12, T21, T22) and four for pedestrians (P11, P12, P21, P22). Paired lights on the same road behave identically, so the controller only needs to drive one set per road.

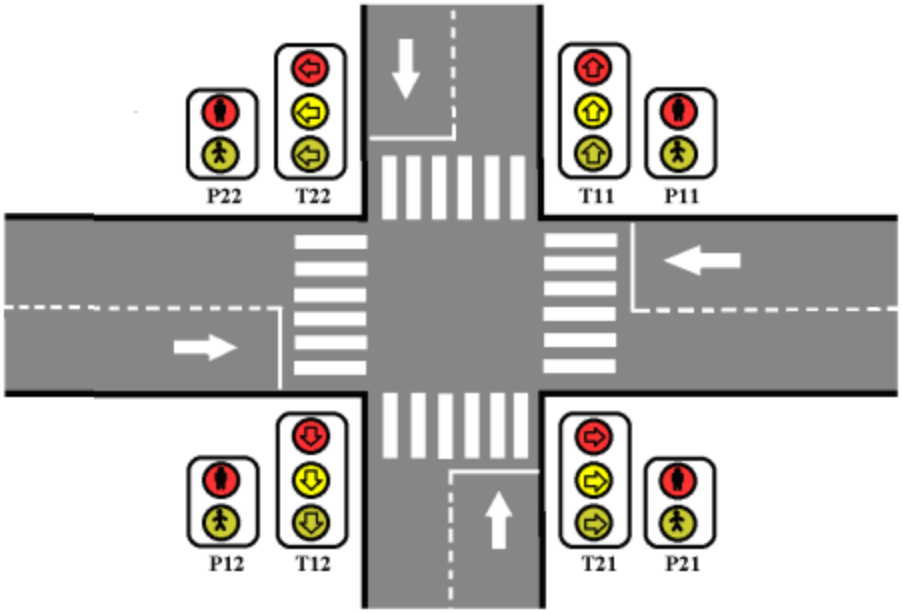

Figure 1: The intersection from Zhilevski et al. [1]

The three operating modes defined by Zhilevski et al. [1] are summarized below.

Normal Mode
The standard timed cycle in which both roads receive equal green time. The vehicle lights cycle through five phases: red, red+yellow, green, flashing green, and yellow. Pedestrian lights are green when the corresponding road's vehicle light is red. The total cycle length is 80 seconds with a 50/50 split between the two roads.

Increased Traffic Mode
An adaptive mode activated during periods of congestion. The cycle length remains 80 seconds, but the red/green split becomes asymmetric. An input signal (xl) selects which road receives priority, and a mode input selects between a 75/25 or 62.5/37.5 duty-cycle ratio. The priority road receives longer green time while the other road's green phase is shortened proportionally. The flashing green phase from normal mode is omitted in this mode.

Emergency Mode
A failsafe mode used when the system is disabled or malfunctioning. All vehicle and pedestrian lights are off except for the yellow signals on both roads, which flash on and off at a fixed rate (every 2 seconds). There is no cycle structure; the flashing continues indefinitely until a different mode is selected.

Methods

Design approach
Each operating mode is implemented as a standalone synthesizable Verilog module containing a behavioral finite state machine and muxed together within a synthesizable top-level module. All sequential logic uses non-blocking assignments in clocked always blocks. All combinational

logic uses blocking assignments in always @(*) blocks with complete case coverage and default assignments to prevent latch inference. State encoding uses localparam or parameter constants with explicit bit widths.

Normal mode FSM

The normal mode uses 3 control inputs of x0, x1, and x2 which typically are all clocks run at divisions of the prior input. So x1 = x0/2 and x2 = x1/2, these inputs are then used to determine which state the system should be on as their overlap results in a consistent performance for the traffic signals. The system starts with T1 being green, before flickering once x1 = 1 and once x0 = 1 it becomes yellow. At x2=1, T1 switches to red as T2 goes through the same stages but with x2 = 1. This results in equal time between both T1 and T2 as T1 will be green while x2 = 0 and red will be while x2 = 1. These durations are controlled by the period of x1 as that dictates the other 2 clocks while still ensuring both T1 and T2 have equal time. We made sure to avoid latches by ensuring all signals got set at the top of their respective blocks to ensure they wouldn't be interrupted as latches.

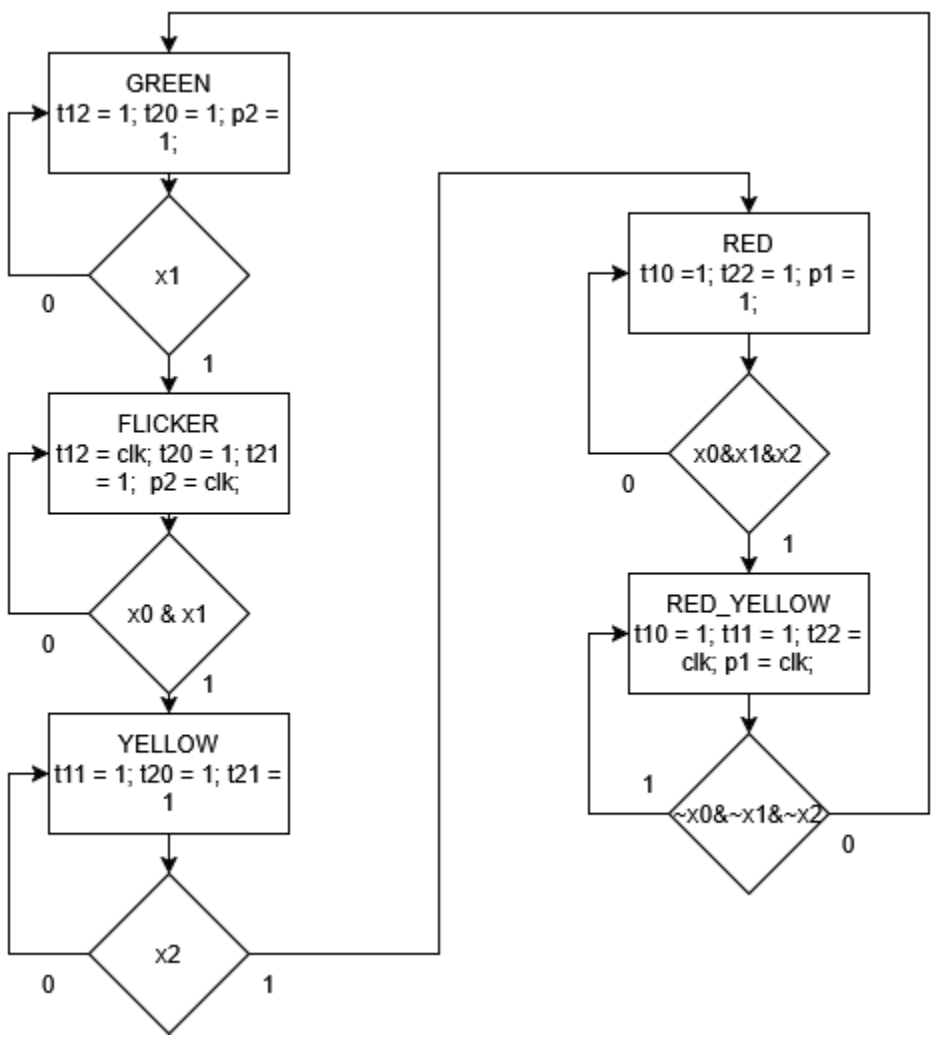

Figure 2: Normal Mode ASM-Chart

Increased traffic mode FSM

The increased traffic mode uses a six-state Moore machine cycling through: S_T1_RY, S_T1_GREEN, S_T1_YELLOW, S_T2_RY, S_T2_GREEN, and S_T2_YELLOW. The red+yellow and yellow phases have fixed two-second durations. The green phase durations are variable, determined by a combinational lookup indexed by the concatenation of the mode and xl inputs. For the 75/25 split, the priority road gets 56 seconds of green and the other gets 16 seconds. For the 62.5/37.5 split, the values are 46 and 26 seconds. These durations account for the 4 seconds of transition phases (red+yellow and yellow) per half-cycle, so the total red time experienced by each road matches the target percentage of the 80-second cycle.

Pedestrian signals are derived directly from the current state. Pedestrians crossing road 1 (P1) receive a walk signal when T1 vehicles are stopped (during T2's active phases), and vice versa. During the red+yellow transitions, both pedestrian signals are held red as a safety measure.

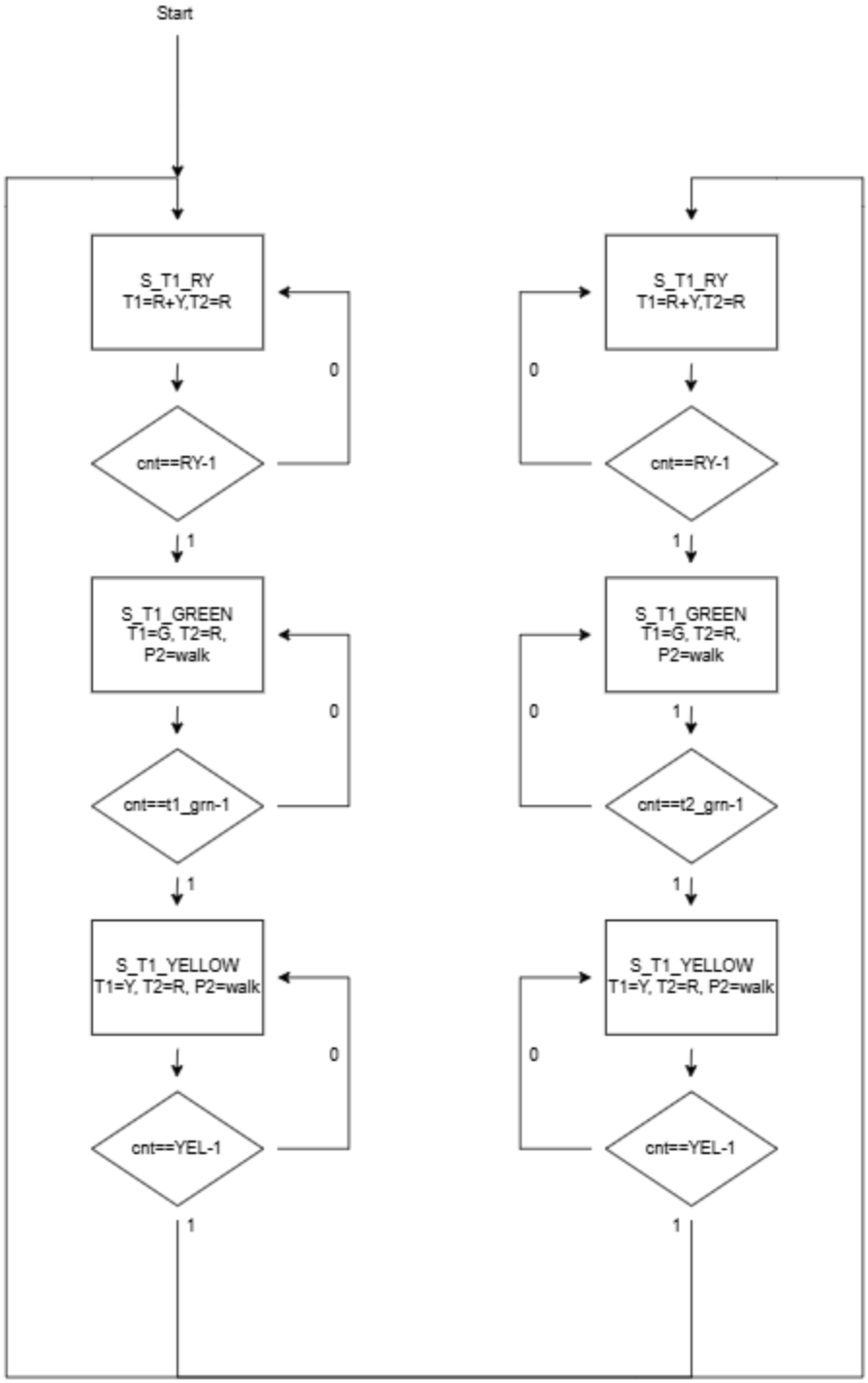

Figure 3: Increased Traffic ASM-Chart

Emergency mode

The emergency mode requires no state machine in the traditional sense. A single counter toggles the yellow outputs for both roads on and off every two clock cycles. All other outputs (red,

green, pedestrian) are held low. On reset, the counter initializes to zero and the yellow signals start in the off state.

Top-level controller

Some of the key aspects of the Top-Level Controller are the cascading registers used to produce x0, x1, and x2 for the Normal mode. Along with that, it holds a counter that increments on x2's edge, this counter is used during a new Autonomous mode where Normal or Traffic are selected based on the priority input. The counter will track how many cycles of the light have been focused on a certain priority and will change between Normal to Low Priority Traffic to High Priority Traffic as the priority stays. This counter also uses a lock, where an input to the controller can be toggled to control whether the counter will increment at all or not, effectively locking the auto mode into whichever state it was most recently in. The reason this counter will trigger on x2's edge is because this will line up the counting with the cycles of the light along with aligning the Normal and Traffic mode so when the modes swap there won't be a sudden jump or any studders. Alongside this Autonomous mode are the individual modes where Normal, Traffic, or Emergency can be selected without switching to maintain control over which mode is active at what time. With this, Emergency has been set as the default mode so should the system fail, it will automatically run the Emergency state signalling an issue. We made sure to avoid latches through having all relevant signals be set at the top of potential cases where they would not be set. This controller unifies control over the different operating modes into a single module along with adding the unique ability to automatically transition between the modes in real time based on inputs.

This top-level integration is the primary structural improvement over the Zhilevski et al. design [1], which treated each mode as a completely separate entity with no mechanism for runtime switching.

```
//Normal_Mode clk management
always @ (negedge clk) begin
   if(rst) begin clk10 <= 0; clk_count <= 0; end
   else begin
      clk_count <= clk_count == 4'd9 ? 0 : clk_count +1;
      clk10 <= clk_count == 4'd9 ? ~clk10 : clk10;
   end
end

always @(negedge clk10 or rst) begin
   if(rst) clk20 <=0;
   else clk20 <= ~clk20;
end

always @(negedge clk20 or rst) begin
   if(rst) clk40 <=0;
   else clk40 <= ~clk40;
end



always @ (negedge clk40 or posedge rst) begin
   if (rst || mode != AU) count <= 3'b000;
   else begin
```

```
        if(xl) begin
            count <= count == 3'b011 || hold_a ? count : count +1;
        end
        else begin
            count <= count == 3'b101 || hold_a ? count : count-1;
        end
    end
end

always @ (count) begin
    auto_n <= 1; mode_t <= 0;
    case(count)
        3'b101: begin auto_n <= 0; mode_t <= 1; end
        3'b110: auto_n <= 0;
        3'b111: auto_n <= 1;
        3'b000: auto_n <= 1;
        3'b001: auto_n <= 1;
        3'b010: auto_n <= 0;
        3'b011: begin auto_n <= 0; mode_t <= 1; end
    endcase
end

// Make a simpler mode so the mux doesn't generate multiple layers
always @ (*)
    set_mode = (mode == 2'b00) ? EM : (mode == 2'b01) ? NO : (mode == 2'b10) ? TR : (mode == 2'b11) ?
((auto_n == 1'b0) ? TR : NO) : 2'b00;
```

Figure 4: Clocking Segment of Top-Level Controller

Synthesis

All modules are synthesized using Synopsys Design Compiler. The synthesis script reads each module, compiles it, and generates area, timing, and power reports.

The basic set of commands we used can be broken into startup which was done to establish the synthesis environment, and then the specific per module instructions used. First is the startup instructions used:

$ setup-synopsys

$ dc_shell

* set link_library {* lsi_10k.db dw_foundation.sldb}

* set target_library {lsi_10k.db}

Next we would run this instruction to set the file/module we were going to synthesize where [filename] was the actual file:

* read_file {[filename]}

Following that, we would begin to compile and run analysis on the module we loaded using these instructions:

* create_clock clk -period 10

* compile

* report_area

* report_timing

* report_power

We would also occasionally use the command set_output_delay for Normal and Traffic to properly define the output timing

Simulation

Verification is performed using Synopsys VCS. The top-level module has a dedicated testbench that exercises all relevant input combinations and verifies correct state-sequencing for all traffic modes. Our simulations featured 2 types of tests, one where we would inspect the waveform to ensure that the pattern and structure of the lights was accurate and one using $display outputs to confirm timing and accurate state changes. Through both these tests we were able to ensure that flicking worked along with making sure the lights have their expected time based on normal or traffic mode and priority. This became more important as we tested the top level module, as after ensuring the structure of base modules timing became the most important. We would use these $display to trigger on state changes of our normal and timing modules and we could track the time to ensure our auto feature properly swapped between both the normal and traffic mode along with properly switching the mode of traffic to the highest level.

Within our tb for normal, we mainly focused on making sure that the structure of the waveform matched the expected [1]. This meant we directly drove x1, x2, and x3 to be 10, 20, and 40 ns respectively to get a readable waveform. For the traffic mode tb, we instead focused more on the $display output along with individual tests for the different priorities and modes conducting a full test of all potential settings. Finally for the top level module, we conducted the same tests for normal and traffic modes along with a new test for emergency focusing on testing the flicker. After that we had a test to track the counting of the auto feature along with using more $display outputs to monitor the internal counter used to determine which auto mode should be used and to ensure the timing actually changed to match the expected priority shift.

```
T=1770  S_T1_RY      | T1=110 | T2=100 | P1=0 P2=0
TRAFFIC Count:          -2
T=1774  S_T1_GREEN   | T1=001 | T2=100 | P1=0 P2=1
TRAFFIC Count:          -2
T=1826  S_T1_YELLOW  | T1=010 | T2=100 | P1=0 P2=1
TRAFFIC Count:          -2
T=1830  S_T2_RY      | T1=100 | T2=110 | P1=0 P2=0
TRAFFIC Count:          -2
T=1834  S_T2_GREEN   | T1=100 | T2=001 | P1=1 P2=0
TRAFFIC Count:          -2
T=1926  S_T2_YELLOW  | T1=100 | T2=010 | P1=1 P2=0
TRAFFIC Count:          -2
T=1930  S_T1_RY      | T1=110 | T2=100 | P1=0 P2=0
TRAFFIC Count:          -2
T=1934  S_T1_GREEN   | T1=001 | T2=100 | P1=0 P2=1
NORMAL Count:          -1
T=1972  S_T1_GREEN   | T1=001 | T2=100 | P1=0 P2=1
NORMAL Count:          -1
T=2012  S_T1_FLICKER | T1=001 | T2=110 | P1=0 P2=1
NORMAL Count:          -1
T=2032  S_T1_YELLOW  | T1=010 | T2=110 | P1=0 P2=0
NORMAL Count:          -1
T=2052  S_T1_RED     | T1=100 | T2=001 | P1=1 P2=0
NORMAL Count:          -1
T=2112  S_T1_RED_YELLOW | T1=110 | T2=001 | P1=1 P2=0
NORMAL Count:           0
T=2132  S_T1_GREEN   | T1=001 | T2=100 | P1=0 P2=1
NORMAL Count:           0
T=2172  S_T1_FLICKER | T1=001 | T2=110 | P1=0 P2=1
NORMAL Count:           0
T=2192  S_T1_YELLOW  | T1=010 | T2=110 | P1=0 P2=0
```

Figure 5: Simulation Results from Autonomous Mode

Results

Simulation results

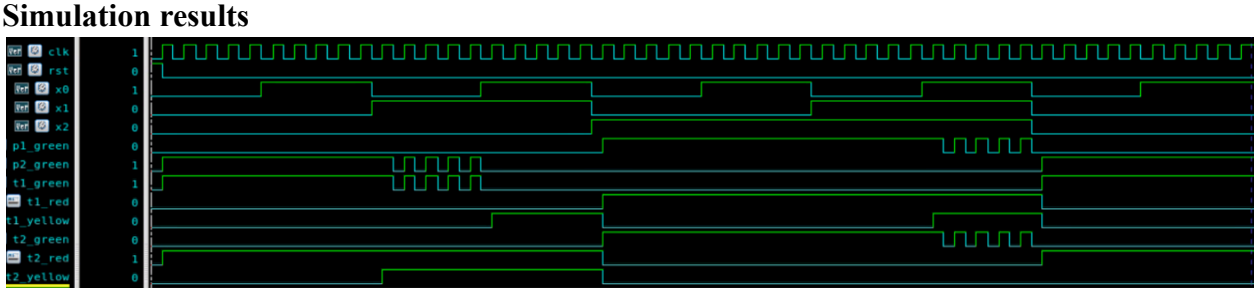

Figure 6: Normal Mode Waveform

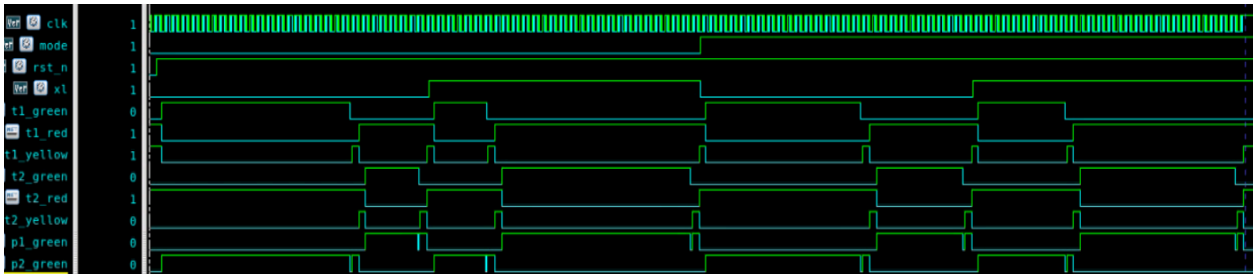

Figure 7: Traffic Mode Waveform

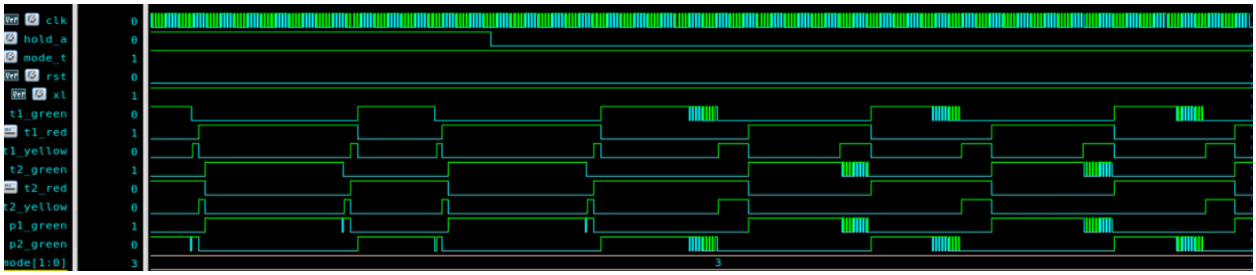

Figure 8: Auto Mode Waveform

In the Normal Mode Waveform (Figure 6) we can see the structure of a single cycle, this matches what is desired [1]. This will be the base timing structure used in the future implementations of Traffic and Auto mode as it keeps T1 and T2 active for equal time, seen clearly by T1 staying red while x2=1 and T2 staying red while x2 = 0. This shows states are progressing as expected.

Next inside of the traffic mode (Figure 7), we have a few differences in how the yellow phase works, as here it only stays for a single clock period giving more time for lights to be green. Within that, this section of the waveform highlights 4 different tests of the Traffic mode, where the priority gets flipped in between cycles and after 2 cycles the mode gets switched to have lower priority. This can be seen in the vastly different length of T2_Green lights across the various times it is active. Going from smallest, to largest, to 2nd smallest, to 2nd largest. This shows the same progression discussed above showcasing that both the mode and priority switching are active along with showcasing our different states and state transitions flowing smoothly.

To note the function of the auto mode (Figure 8), within the waveform you can tell that the % time that the lights are either red or green does change across the various cycles. This matches the setup of the section as the count moves from -2 to 1, meaning it starts with T2 priority before moving into normal priority. Specifically, T1 goes from being active for around 60ns to 80ns, matching our desired change done automatically within the top control module. On top of this, it displays a clear transition as the light doesn’t stutter or jump while switching between the Normal module and the Traffic module. The switch can also be highlighted by the different length in yellow light times.

Synthesis results

Area Reports	
Normal	Number of ports: 13 Number of nets: 24 Number of cells: 17 Number of combinational cells: 14 Number of sequential cells: 3 Number of macros/black boxes: 0 Number of buf/inv: 4 Number of references: 10 Combinational area: 22.000000 Buf/Inv area: 4.000000 Noncombinational area: 21.000000 Macro/Black Box area: 0.000000 Net Interconnect area: undefined (No wire load specified) Total cell area: 43.000000 Total area: undefined
Traffic	Number of ports: 12 Number of nets: 175 Number of cells: 152 Number of combinational cells: 133 Number of sequential cells: 19 Number of macros/black boxes: 0 Number of buf/inv: 24 Number of references: 27 Combinational area: 218.000000 Buf/Inv area: 25.000000 Noncombinational area: 167.000000 Macro/Black Box area: 0.000000 Net Interconnect area: undefined (No wire load specified) Total cell area: 385.000000 Total area: undefined
Emergency	Number of ports: 10 Number of nets: 12 Number of cells: 10 Number of combinational cells: 2 Number of sequential cells: 2 Number of macros/black boxes: 0 Number of buf/inv: 0 Number of references: 2 Combinational area: 2.000000 Buf/Inv area: 0.000000 Noncombinational area: 14.000000 Macro/Black Box area: 0.000000 Net Interconnect area: undefined (No wire load specified) Total cell area: 16.000000
Controller	Number of ports: 50 Number of nets: 339 Number of cells: 273 Number of combinational cells: 229 Number of sequential cells: 34 Number of macros/black boxes: 0 Number of buf/inv: 50 Number of references: 27 Combinational area: 413.000000 Buf/Inv area: 54.000000 Noncombinational area: 284.000000 Macro/Black Box area: 0.000000 Net Interconnect area: undefined (No wire load specified) Total cell area: 697.000000 Total area: undefined

Table 1: Area Synthesis Reports

Timing Reports

Normal	
Traffic	
Emergency	
Controller	

```
Point                                        Incr       Path
-----------------------------------------------------------------
clock clk (fall edge)                        5.00       5.00
clk (in)                                     0.00       5.00 f
U8/Z (AO7)                                   1.04       6.04 r
U26/Z (IVP)                                  0.16       6.20 f
p1 (out)                                     0.00       6.20 f
data arrival time                                       6.20

clock clk (rise edge)                       10.00      10.00
clock network delay (ideal)                  0.00      10.00
output external delay                       -2.00       8.00
data required time                                      8.00
-----------------------------------------------------------------
data required time                                      8.00
data arrival time                                      -6.20
-----------------------------------------------------------------
slack (MET)                                             1.80
```

```
Point                                        Incr       Path
-----------------------------------------------------------------
clock clk (rise edge)                        0.00       0.00
clock network delay (ideal)                  0.00       0.00
t2_grn_dur_reg[2]/CP (FD2P)                  0.00       0.00 r
t2_grn_dur_reg[2]/QN (FD2P)                  2.02       2.02 f
U117/Z (ND2)                                 0.64       2.65 r
U179/Z (ND2)                                 0.22       2.87 f
U180/Z (ND2)                                 0.64       3.51 r
U120/Z (EO)                                  1.13       4.63 f
U187/Z (ND3)                                 1.07       5.71 r
U147/Z (ND2)                                 0.22       5.92 f
U129/Z (AN2P)                                0.94       6.87 f
U128/Z (AN3)                                 0.97       7.83 f
U222/Z (AO1P)                                1.31       9.15 r
cnt_reg[3]/D (FD2)                           0.00       9.15 r
data arrival time                                       9.15

clock clk (rise edge)                       10.00      10.00
clock network delay (ideal)                  0.00      10.00
cnt_reg[3]/CP (FD2)                          0.00      10.00 r
library setup time                          -0.85       9.15
data required time                                      9.15
-----------------------------------------------------------------
data required time                                      9.15
data arrival time                                      -9.15
-----------------------------------------------------------------
slack (MET)                                             0.00
```

```
Point                                        Incr       Path
-----------------------------------------------------------------
clock clk (rise edge)                        0.00       0.00
clock network delay (ideal)                  0.00       0.00
t11_reg/CP (FD1)                             0.00       0.00 r
t11_reg/Q (FD1)                              1.42       1.42 f
U10/Z (NR2)                                  0.81       2.23 r
t11_reg/D (FD1)                              0.00       2.23 r
data arrival time                                       2.23

clock clk (rise edge)                       10.00      10.00
clock network delay (ideal)                  0.00      10.00
t11_reg/CP (FD1)                             0.00      10.00 r
library setup time                          -0.80       9.20
data required time                                      9.20
-----------------------------------------------------------------
data required time                                      9.20
data arrival time                                      -2.23
-----------------------------------------------------------------
slack (MET)                                             6.97
```

```
Point                                        Incr       Path
-----------------------------------------------------------------
clock clk' (rise edge)                       5.00       5.00
clock network delay (ideal)                  0.00       5.00
clk10_reg/CP (FDS2)                          0.00       5.00 r
clk10_reg/Q (FDS2)                           1.40       6.40 f
dut_n/x1 (normal)                            0.00       6.40 f
dut_n/U9/Z (IVA)                             0.31       6.71 r
dut_n/U12/Z (AN4P)                           1.23       7.94 r
dut_n/current_state_reg[0]/D (FD1)           0.00       7.94 r
data arrival time                                       7.94

clock clk (rise edge)                       10.00      10.00
clock network delay (ideal)                  0.00      10.00
dut_n/current_state_reg[0]/CP (FD1)          0.00      10.00 r
library setup time                          -0.80       9.20
data required time                                      9.20
-----------------------------------------------------------------
data required time                                      9.20
data arrival time                                      -7.94
-----------------------------------------------------------------
slack (MET)                                             1.26
```

Table 2: Timing Synthesis Reports

Power Reports	
Normal	

```
                  Internal         Switching          Leakage            Total
Power Group       Power            Power              Power              Power
 (   %    )  Attrs
--------------------------------------------------------------------------------
io_pad              0.0000           0.0000            0.0000               NA
(     N/A)
memory              0.0000           0.0000            0.0000               NA
(     N/A)
black_box           0.0000           0.0000            0.0000               NA
(     N/A)
clock_network       0.0000         299.9191            0.0000               NA
(     N/A)  i
register            0.0000          97.3969            0.0000               NA
(     N/A)
sequential          0.0000           0.0000            0.0000               NA
(     N/A)
combinational       0.0000         127.2408            0.0000               NA
(     N/A)
--------------------------------------------------------------------------------
Total               0.0000 nW      524.5568 nW         0.0000               NA
```

Traffic	

```
                  Internal         Switching          Leakage            Total
Power Group       Power            Power              Power              Power
 (   %    )  Attrs
--------------------------------------------------------------------------------
io_pad              0.0000           0.0000            0.0000               NA
(     N/A)
memory              0.0000           0.0000            0.0000               NA
(     N/A)
black_box           0.0000           0.0000            0.0000               NA
(     N/A)
clock_network       0.0000         260.6087            0.0000               NA
(     N/A)  i
register            0.0000        1.3862e+03           0.0000               NA
(     N/A)
sequential          0.0000           0.0000            0.0000               NA
(     N/A)
combinational       0.0000        2.2538e+03           0.0000               NA
(     N/A)
--------------------------------------------------------------------------------
Total               0.0000 nW     3.9006e+03 nW        0.0000               NA
```

Emergency	

```
                  Internal         Switching          Leakage            Total
Power Group       Power            Power              Power              Power
 (   %    )  Attrs
--------------------------------------------------------------------------------
io_pad              0.0000           0.0000            0.0000               NA
(     N/A)
memory              0.0000           0.0000            0.0000               NA
(     N/A)
black_box           0.0000           0.0000            0.0000               NA
(     N/A)
clock_network       0.0000           0.0000            0.0000               NA
(     N/A)  i
register            0.0000         268.2343            0.0000               NA
(     N/A)
sequential          0.0000           0.0000            0.0000               NA
(     N/A)
combinational       0.0000         281.1279            0.0000               NA
(     N/A)
--------------------------------------------------------------------------------
Total               0.0000 nW      549.3622 nW         0.0000               NA
```

Controller	

```
                  Internal         Switching          Leakage            Total
Power Group       Power            Power              Power              Power
 (   %    )  Attrs
--------------------------------------------------------------------------------
io_pad              0.0000           0.0000            0.0000               NA
(     N/A)
memory              0.0000           0.0000            0.0000               NA
(     N/A)
black_box           0.0000           0.0000            0.0000               NA
(     N/A)
clock_network       0.0000        2.4016e+03           0.0000               NA
(     N/A)  i
register            0.0000        2.4252e+03           0.0000               NA
(     N/A)
sequential          0.0000         855.1407            0.0000               NA
(     N/A)
combinational       0.0000        6.1493e+03           0.0000               NA
(     N/A)
--------------------------------------------------------------------------------
Total               0.0000 nW     1.1831e+04 nW        0.0000               NA
```

Table 3: Power Synthesis Reports

To start, we can see that the Controller requires a 10ns clock, shown by the limited slack left and the fact the traffic module individually has no slack left. We can also see that it has a cell area of 697nm which is mostly made up of Combinational logic. From our output we can see that the Traffic module is the main constraining device within the Controller top module with it showing many similar patterns with the other modules, however on a much larger scale. This can be seen in Power where Traffic has a magnitude larger nW usage compared to Emergency and Normal.

On top of that, Traffic is the critical path restricting the Timing of the whole system as it barely passes the timing report with a period 10ns clock and a resulting slack of 0. Again, in Area the Traffic module represents the vast majority in the Controller module, being much larger than the other 2 individual modules. However, across all of the modules we can see that combinational logic takes up the vast majority of Area and Power with registers and clock being the one ones close, or surpassing in the case of Normal Power. With that in mind, going forward a further improvement could be a reduction in combinational logic to improve the power loss caused and potentially reduce some area. Alongside that, finding an alternate control for the Normal module would help reduce the power loss due to the clock network. Overall, finding improvements in the increased traffic mode is key as that represents the majority of Area, Power, and Timing within the system where it even defines the clock time due to timing constraints.

Discussion

The behavioral FSM approach offers several concrete advantages over the gate-level design in [1]. First, explicit state encoding with localparam constants and a case statement makes the state transitions readable and verifiable by inspection. The Zhilevski design relies on the interaction of combinational primitives to implicitly define states, which requires tracing signal values through multiple gate levels to understand the current operating phase.

Second, the separation of sequential and combinational logic into distinct always blocks, with default assignments covering all outputs in every case branch, guarantees that no latches are inferred during synthesis. The gate-level approach in [1] does not use always blocks at all, instead relying on continuous assignments and primitive instantiations, which sidesteps the latch issue but at the cost of design flexibility.

Third, the parameterized timing in the increased traffic mode allows the duty-cycle ratios to be adjusted by changing localparam values rather than redesigning the combinational logic. The Zhilevski design required separate Verilog source files for the 62.5/37.5 and 75/25 modes, duplicating most of the logic.

The top-level wrapper with runtime mode selection is completely absent from the original design. Our implementation demonstrates a more practical integration that would be necessary in any real deployment of a traffic light controller.

One of the biggest tradeoffs comes in the form of the Traffic module, as it is the main restricting component that causes the Controller to be so large and require a slower clock. The Traffic module's individual reports show it is nearly 40% of Power consumption, 55% of Area, and the critical path in Timing. Along with that by adding together all the totals of the reports we can see the cost of the mux overhead and clock management that comes from the Controller. 444 out 697nm are made up of individual modules, meaning the muxes and clock registers are 253nm, still making the Controller smaller than the Traffic module. In terms of power, the Controller components represent around 50% of all power consumed, which is significant as the majority comes from the additional computational logic added through the muxes alongside the clock network which is the cost of running the Normal module signals through the Controller. White there is a clear cost in terms of power for running the Controller, it still seems the Traffic module is the biggest room for improvement especially within timing.

Our implementation is inherently a serial design. Because a FSM like this cannot be pipelined, and considering we can only have one mode active at once, techniques that evoke parallelism do not make much sense.

Conclusion

This project implemented a traffic light controller in synthesizable behavioral Verilog HDL, covering three operating modes: normal, increased traffic, and emergency. Each mode was designed as an independent FSM and integrated under a top-level module with runtime mode selection. The design improves upon the reference implementation by Zhilevski et al. [1] through the use of explicit state machines, parameterized timing, latch-free combinational logic, and a unified controller. Simulation confirmed correct functionality across all modes and input configurations, and synthesis with the lsi_10k.db and dw_foundation.sldb produced a clean netlist with no warnings.

References

[1] M. Zhilevski, M. Mikhov, and M. Zhilevska, "Design of a Traffic Light System Using Verilog HDL," in 2023 5th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), Istanbul, Turkiye, 2023, pp. 1-6, doi: 10.1109/HORA58378.2023.10156804.